Development Workflow and Configuration Plan – SESCAM Radiodiagnostics

Software Engineering II – Course Project 2026/2027. Team of 6. Draft v0.1 for team review.

1. Purpose and Scope

This document defines a consistent development workflow, the technical setup steps, and a small configuration test to validate both before implementation starts. It covers technical preparation and project planning (Block 1). It does not define the system architecture or the module list, which will be decided in later iterations and recorded in DECISIONS.md.

2. Guiding Principles (from the course specification)

- **Unified Process:** use-case driven, iterative and incremental, architecture-centric.
- **Human in the loop:** the engineer designs interfaces and architecture; the AI agent implements contracts and runs tests. Accountability stays with each human team member.
- **Traceability:** every decision made in a meeting becomes a GitHub Issue, tracked on a GitHub Projects Kanban board and delivered through a Pull Request.
- **Git Flow:** no direct commits to main; all mainline integration goes through peer-reviewed Pull Requests.
- **Semantic Versioning 2.0.0** for every component, mirrored in Git Flow branches and tags.
- **GitHub as primary evidence:** wiki, commits, issues, board and Insights are what gets evaluated. Tooling outside GitHub is avoided unless justified in DECISIONS.md.
- **No overengineering:** every tool or technology added must be justified in writing.

3. Proposed Development Workflow

3.1 Branching model (Git Flow)

Branch	Purpose	Created from	Merged into
`main`	Released, tagged versions only	–	–
`develop`	Integration of finished work	`main`	`release/*`
`feature/<issue-id>-<short-name>`	One task or use case	`develop`	`develop` (via PR)
`release/<x.y.z>`	Stabilisation of a release	`develop`	`main` and `develop`
`hotfix/<x.y.z>`	Urgent fix on a release	`main`	`main` and `develop`

Rules:

- `main` and `develop` are protected: no direct pushes, at least one approving review, CI must pass before merge.
- Branch names are lowercase, in English, with hyphens (example: `feature/12-appointment-module`).
- Feature branches are short-lived and deleted after merge.

3.2 From meeting to delivery (traceability chain)

1. The team meets and writes minutes in the wiki (decisions and deliverables listed).
2. Each decision or deliverable becomes a GitHub Issue with an owner, a label (requirement or task type), an iteration and a milestone.
3. The Issue is placed on the Kanban board (columns: Backlog, Ready, In progress, In review, Done).
4. The owner creates a `feature/*` branch from `develop` and works on it.
5. The owner opens a Pull Request into `develop` that references the Issue (for example `Closes #12`).

6. A teammate reviews. CI must be green. The PR is merged with the agreed strategy (squash merge recommended to keep history readable).
7. The Issue closes automatically and the board updates.

3.3 Pull Request rules

- One PR covers one Issue and stays small enough to review in about 30 minutes.
- The author never merges their own PR.
- The reviewer checks: the PR matches the Issue, interfaces are respected, tests exist and pass, no unjustified dependencies, no secrets, documentation updated.
- AI-generated code is reviewed with the same rigour as human code. The PR description states whether an agent was used and for what.
- Review comments are resolved before merge.

3.4 Commit conventions

- Language: English, imperative mood.
- Format (Conventional Commits): `type(scope): summary`, for example `feat(appointments): add booking interface`.
- Types: `feat`, `fix`, `docs`, `test`, `refactor`, `build`, `ci`, `chore`.
- Reference the Issue number in the body or footer.

3.5 Versioning and releases

- Each Maven module has its own SemVer version, planned in advance as milestones (example: Appointment Management Module v1.3.0).
- Releases are cut from `release/<x.y.z>` and tagged on `main` as `v<x.y.z>`.
- A component changelog (CHANGELOG.md per module) lists what changed in each version.
- The final transition iteration integrates all components into one tagged build.

3.6 Responsibilities: humans and agents

Activity	Human engineers	AI agent (OpenCode)
Requirements elicitation, use case prioritisation	Yes	No
Use case analysis and design, architecture decisions	Yes	Can suggest alternatives only
Interface design (Java interfaces, module boundaries)	Yes	Can audit, not decide
Unit and integration test design	Yes	Can scaffold from the design
Boilerplate and contract-compliant implementation	Reviews	Yes
Test execution	Reviews results	Yes
Code review and quality audit	Yes	Can assist
Planning, estimates, schedule	Yes	Can produce a forecast to compare against

3.7 AI agent rules (OpenCode)

Shared files in the repository root, versioned like code and changed only through PRs:

- **AGENTS.md:** persistent rules for the agent (architecture constraints, coding conventions, what it must not do, tooling allowed).
- **CONTEXT.md:** mapping of functional requirements to use cases and modules, domain glossary.
- **PLAN.md:** current iteration scope and task breakdown.
- **DECISIONS.md:** every assumption and design decision with its justification and date.

Minimum rules for AGENTS.md:

1. Do not add dependencies, frameworks or tools that are not listed in DECISIONS.md.
2. Implement only against the interfaces provided; do not change interfaces without a human decision.

3. Respect module boundaries: modules communicate only through interface packages.
4. Work only inside the current iteration scope described in PLAN.md.
5. Never commit to `main` or `develop`; work on a feature branch and open a PR.
6. Use only fictitious data, never real patient data or credentials.

3.8 Definition of Done (per task)

- Code compiles and all tests pass in CI.
- At least one teammate approved the PR.
- Issue linked and closed, board updated.
- Documentation, CHANGELOG and DECISIONS.md updated when relevant.
- No unresolved review comments.

4. Required Technical Setup Steps

4.1 Repository and GitHub (once, by the repository administrator)

1. Create the repository under the institutional UCLM student account.
2. Add all 6 team members and the laboratory instructor's GitHub account as collaborators.
3. Enable Issues, Projects and Wiki.
4. Create `develop` from `main` and set `develop` as the default branch.
5. Configure branch protection for `main` and `develop`: require pull request, at least 1 approval, passing status checks, no direct pushes. Verify availability for the repository type (private repositories may need the GitHub Campus benefits).
6. Create labels (for example `requirement`, `use-case`, `bug`, `docs`, `ci`, `iteration-0`, `iteration-N`).
7. Create the Kanban board with the five columns and link it to the repository.
8. Create milestones for planned releases (SemVer).
9. Add `.gitignore` (Java, Maven, Eclipse, VS Code) and a README with the wiki link.
10. Create the wiki home page with an index of planning documents, minutes, models and spreadsheets. Reserve the final section "Self-Assessment and Experience".

4.2 Local environment (every team member)

1. Git installed, name and email configured; SSH key or credential manager set up for GitHub.
2. JDK installed. Version to be confirmed in DECISIONS.md (17 is used as a placeholder in the test below).
3. Apache Maven installed (`mvn -v` works).
4. Eclipse (course-required IDE) with Maven support, and Visual Paradigm.
5. OpenCode installed and configured with the team's chosen model. Agree on the same AGENTS.md for everyone.
6. Clone the repository and check out `develop`.

4.3 Repository skeleton

```
repo/
  AGENTS.md
  CONTEXT.md
  PLAN.md
  DECISIONS.md
  README.md
  pom.xml                 (parent, packaging pom)
  module-a/
  module-b/
  .github/workflows/ci.yml
```

4.4 Continuous integration

A GitHub Actions workflow that compiles and runs tests on every PR into `develop` and `main`. Mark it as a required status check in branch protection once it has run once.

5. Small Configuration Test

5.1 Goal

Prove end to end that the team can create a feature branch, open a PR, pass CI, get a review, merge into `develop`, and that Maven multi-module builds work on every machine. Everything uses generic module names, to be replaced with real ones later.

5.2 Structure

- Parent project `sescam-parent` (packaging `pom`).
- `module-a` exposes an interface `Greeter` in an `api` package.
- `module-b` depends on `module-a` and uses only the interface.

5.3 Parent pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>es.uclm.sescam</groupId>
  <artifactId>sescam-parent</artifactId>
  <version>0.1.0-SNAPSHOT</version>
  <packaging>pom</packaging>

  <modules>
    <module>module-a</module>
    <module>module-b</module>
  </modules>

  <properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>5.10.2</version>
        <scope>test</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>

  <build>
    <pluginManagement>
      <plugins>
        <plugin>
          <artifactId>maven-surefire-plugin</artifactId>
          <version>3.2.5</version>
```

```
        </plugin>
      </plugins>
    </pluginManagement>
  </build>
</project>
```

5.4 Module poms (summary)

- `module-a/pom.xml`: parent `sescam-parent`, artifactId `module-a`, test dependency on `junit-jupiter`.
- `module-b/pom.xml`: parent `sescam-parent`, artifactId `module-b`, dependency on `module-a` (same version) and test dependency on `junit-jupiter`.

5.5 Code (summary)

- `module-a`: interface `es.uclm.sescam.modulea.api.Greeter` with `String greet(String name)`, and an implementation class in a separate `internal` package.
- `module-b`: a class that receives a `Greeter` and calls it, plus one JUnit 5 test that uses a simple implementation of the interface and asserts the result.

5.6 GitHub Actions workflow (.github/workflows/ci.yml)

```
name: CI

on:
  pull_request:
    branches: [develop, main]
  push:
    branches: [develop]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
```

```
          java-version: '17'
          cache: maven
      - name: Build and test
        run: mvn -B verify
```

5.7 Test procedure

1. The administrator pushes the skeleton, the parent pom, both modules and the workflow to `develop` (the only allowed initial push, before protection is enabled).
2. Enable branch protection and require the CI check.
3. Developer 1 creates an Issue "Add module-b test" and a branch `feature/1-module-b-test`.
4. Developer 1 changes the test, runs `mvn -B verify` locally and pushes.
5. Developer 1 opens a PR into `develop` with `Closes #1`.
6. CI runs and passes. Developer 2 reviews and approves. Developer 1 does not merge their own PR; another member merges.
7. Verify that the Issue closed and the board moved it to Done.
8. Negative test: open a PR with a deliberately failing test and confirm the merge button is blocked. Try a direct push to `develop` and confirm it is rejected.
9. Release rehearsal: create `release/0.1.0` from `develop`, merge into `main`, tag `v0.1.0`, merge back into `develop`.
10. Every other team member clones and runs `mvn -B verify` successfully on their own machine.

5.8 Success criteria

- Multi-module build passes locally for all 6 members and in CI.
- Direct pushes to protected branches are rejected.
- A failing build blocks merge.
- The full chain Issue, branch, PR, review, merge, closed Issue, board update works.
- The tag `v0.1.0` exists on `main` and `develop` contains the release changes.
- An OpenCode session reads AGENTS.md and produces a change on a feature branch that is reviewed like any other PR.

6. Decisions and Open Items for DECISIONS.md

Item	Status	Note
Source control and tracking	Decided (proposed)	GitHub only: Issues, Projects, Wiki, Actions
Azure DevOps	Proposed: not used	Duplicates GitHub traceability required by the specification
Azure (cloud hosting)	Open	Optional for the transition iteration (deployment demo); fictitious data only
Documentation language	Decided	English
AI agent environment	Decided	OpenCode (model to be agreed)
Java version	Open	17 used as placeholder
Framework and database	Open	Not needed for the configuration test
Client type (web, desktop, both)	Open	
Maven module list	Open	Generic modules until the architecture is designed
Merge strategy	Proposed	Squash merge into `develop`

7. Next Steps

1. Team reviews and amends this draft in a meeting; minutes go to the wiki.
2. Administrator completes section 4.1.
3. Run the configuration test (section 5) and record the result in the wiki.
4. Write AGENTS.md, CONTEXT.md, PLAN.md and DECISIONS.md.
5. Start Iteration 0 planning.